

Systemy Sztucznej Inteligencji

Dokumentacja projektu – Klasyfikacja przedziału cenowego rosyjskich mieszkań

Kamil Skop, Michał Kosiec, Szymon Sobelski - gr. 8, ?, ?

13 czerwca 2026

Spis treści

1	Część I	2
1.1	Opis programu	2
1.2	Instrukcja obsługi	3
1.3	Opis obsługi	3
2	Część II	4
2.1	Opis działania	4
2.1.1	Przetwarzanie wstępne i binning cen	4
2.1.2	Algorytm KNN	4
2.1.3	Naiwny Bayes	5
2.1.4	Drzewo decyzyjne	5
2.2	Algorytm	6
2.3	Baza danych	7
2.4	Struktura rekordu	7
2.5	Implementacja	8
2.6	Eksperymenty	9
2.7	Wnioski	9
3	Pełen kod aplikacji	10

1 Część I

1.1 Opis programu

Celem projektu jest klasyfikacja przedziału cenowego rosyjskich mieszkań na podstawie ich cech (powierzchnia, liczba pokoi, pietro, lokalizacja geograficzna, typ budynku itp.). Dane wejściowe stanowi plik CSV zawierający 200 000 rekordów. Cena każdego mieszkania zostaje zakwalifikowana do jednego z $N = 5$ równo licznych przedziałów cenowych (klas C1–C5) wyznaczanych dynamicznie kwantylami ze zbioru treningowego.

Program losowo, z użyciem stałego `seed = 42` dzieli wczytane dane na zbiór treningowy oraz testowy w proporcji 80:20 po uprzednim oczyszczeniu go z outlierów na podstawie reguły 3-sigma na cechach ciągłych (gdy odległość dowolnej cechy od średniej przekracza 3 odchylenia standardowe, usuwany jest cały rekord).

Program implementuje od zera, bez użycia zewnętrznych bibliotek uczenia maszynowego, trzy klasyfikatory:

- **KNN** (k najbliższych sąsiadów),
- **Naiwny Bayes** (mieszany: Gauss + Bernoulli + wielomianowy),
- **Drzewo decyzyjne** (kryterium Gini, ograniczenie głębokości i minimalnego listka).

W sprawozdaniu przeprowadzono również analizę rezultatów każdego z wytrenowanych klasyfikatorów.

1.2 Instrukcja obsługi

Wymagania:

- Python 3.12+
- Biblioteka `matplotlib`
- Obecność przetworzonej bazy danych z Kaggle

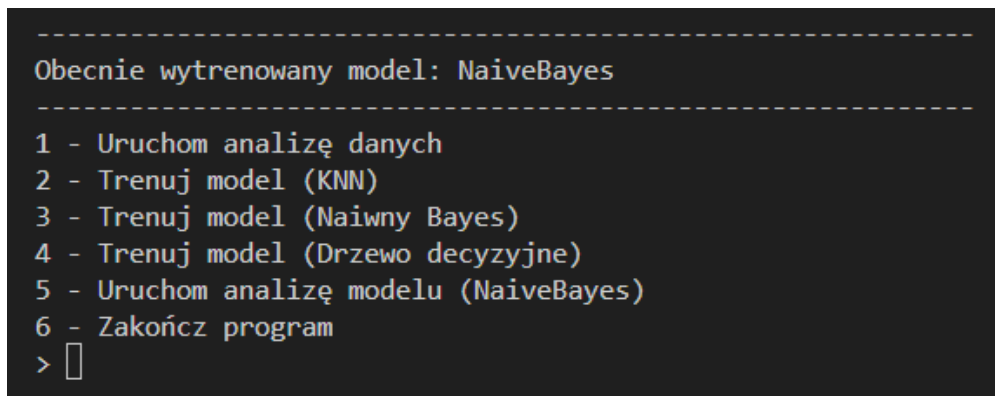
Uruchomienie:

- Upewnić się, że w katalogu projektu znajduje się plik `filtered_v3.csv`. Jeśli ten nie występuje, należy pobrać zbiór danych ze strony Kaggle, a następnie wygenerować go za pośrednictwem skryptu, umieszczonego w katalogu `datalab/`.
- Uruchomić skrypt za pomocą polecenia `python main.py`.

Uwaga: Przetestowanie KNN może wymagać zmniejszenia parametru `EXPECTED_ROWS` w kodzie źródłowym do np. 10 000, ponieważ algorytm ten jest bardzo niewydajny i zwyczajnie nie radzi sobie z większymi ilościami rekordów.

1.3 Opis obsługi

Program wczytuje plik `filtered_v3.csv` z bieżącego katalogu, usuwa outliery i prezentuje menu tekstowe. Należy wybrać interesującą nas opcję, wprowadzając ją z klawiatury i wciskając enter:



```
-----  
Obecnie wytrenowany model: NaiveBayes  
-----  
1 - Uruchom analizę danych  
2 - Trenuj model (KNN)  
3 - Trenuj model (Naiwny Bayes)  
4 - Trenuj model (Drzewo decyzyjne)  
5 - Uruchom analizę modelu (NaiveBayes)  
6 - Zakończ program  
> █
```

Rysunek 1: Zrzut ekranu menu głównego.

1. Analiza danych (wyświetla analizę danych oraz generuje wykresy do katalogów `output_raw/` i `output_clean/`).
2. Trening modelu KNN.
3. Trening modelu Naiwny Bayes.
4. Trening modelu Drzewo decyzyjne.
5. Analiza wyników aktualnie wytrenowanego modelu (macierz pomyłek, dokładność itd.).
6. Wyjście.

2 Część II

2.1 Opis działania

2.1.1 Przetwarzanie wstępne i binning cen

Cena mieszkania jest wartością ciągłą. Aby problem stał się klasyfikacją, ceny dzielone są na N równo licznych przedziałów (kwantylowych):

$$q_i = \text{kwantyl}\left(\frac{i}{N}\right), \quad i = 1, \dots, N - 1.$$

Dla $N = 5$ otrzymujemy 5 klas C1–C5. Podział wyznaczany jest raz z całego zbioru (po usunięciu outlierów), co gwarantuje równo liczne klasy i eliminuje wpływ wartości ekstremalnych na granice.

Usuwanie outlierów – reguła 3-sigma

Dla każdej cechy liczbowej x_j wyznaczana jest średnia μ_j i odchylenie standardowe σ_j . Rekord uznaje się za outlier, jeżeli dla dowolnej cechy j :

$$|x_j - \mu_j| > 3\sigma_j.$$

2.1.2 Algorytm KNN

Klasyfikator k najbliższych sąsiadów ($k = 7$) przydziela klasę na podstawie ważonego głosowania sąsiadów. Dla pary przykładów a, b stosuje się metrykę hybrydową:

$$d(a, b) = \underbrace{\sqrt{\sum_j \left(\frac{a_j - b_j}{s_j}\right)^2}}_{\text{część ciągła}} + d_{\text{region}} + d_{\text{budynek}} + d_{\text{nowe}}$$

gdzie s_j to odchylenie standardowe j -tej cechy ciągłej (normalizacja), a dodatkowe składniki binarne karzą za różny region (+1,0), typ budynku (+0,5) lub różny status nowego budownictwa (+0,4).

Waga sąsiada odwrotnie proporcjonalna do odległości:

$$w_i = \frac{1}{d_i + \varepsilon}, \quad \varepsilon = 10^{-9}.$$

2.1.3 Naiwny Bayes

Klasyczny Naiwny Bayes zakłada, że wszystkie cechy są warunkowo niezależne i mają rozkład normalny. Diagnoza wizualna rozkładów cech wykazała, że założenie o gaussowskości jest niespełnione dla większości zmiennych:

- **Lokalizacja geograficzna:** rozkład wielomodalny (skupiska miejskie).
- **Miesiąc:** rozkład cykliczny.
- **Liczba pokoi / pięter:** wartości dyskretne ze skokami.

W odpowiedzi wdrożono *mieszany Naiwny Bayes* (Mixed Naive Bayes) z trzema niezależnymi torami prawdopodobieństwa:

1. **Gaussowski** – dla $\log(\text{area} + \varepsilon)$ i $\log(\text{kitchen_area} + \varepsilon)$:

$$P(x_j | c) = \frac{1}{\sqrt{2\pi\sigma_{jc}^2}} \exp\left(-\frac{(x_j - \mu_{jc})^2}{2\sigma_{jc}^2}\right).$$

2. **Bernoulliego** – dla cechy binarnej `new_building` (wygładzanie Laplace'a, $\alpha = 1$):

$$P(x_j | c) = p_{jc}^{x_j} (1 - p_{jc})^{1-x_j}, \quad p_{jc} = \frac{n_{1jc} + 1}{n_c + 2}.$$

3. **Wielomianowy** – dla pozostałych cech dyskretnych i kategorycznych (rok, miesiąc, piętro, liczba pięter, pokoje, region, typ budynku, hasz geo):

$$P(x_j = v | c) = \frac{n_{vjc} + \alpha}{n_c + \alpha \cdot |V_j|}.$$

Spatial hashing: współrzędne geograficzne zaokrąglane są do 0,1 stopnia (≈ 11 km), co zastępuje kosztowne klasteryzowanie i pozwala traktować lokalizacje jako kategorie.

Klasyfikacja: wybierana jest klasa z maksymalnym logarytmem a posteriori:

$$\hat{c} = \arg \max_c \left[\log P(c) + \sum_j \log P(x_j | c) \right].$$

2.1.4 Drzewo decyzyjne

Drzewo budowane jest metodą zachłanną (CART). Kryterium podziału to nieczystość Gini:

$$\text{Gini}(S) = 1 - \sum_c \left(\frac{|S_c|}{|S|} \right)^2.$$

Najlepszy podział minimalizuje ważoną sumę nieczystości potomków:

$$\text{Gain} = \text{Gini}(S) - \frac{|S_L|}{|S|} \text{Gini}(S_L) - \frac{|S_R|}{|S|} \text{Gini}(S_R).$$

Ograniczenia: maksymalna głębokość = 13, minimalna liczba próbek w liściu = 10. Trening odbywa się na losowej próbce 5000 rekordów (seed = 42).

2.2 Algorytm

Ponizej przedstawiono pseudokod glownego potoku przetwarzania danych.

Data: Plik CSV z danymi mieszkam, liczba klas $N = 5$, proporcja treningu $r = 0,8$

Result: Wytrenowany klasyfikator, wyniki na zbiorze testowym

Wczytaj wszystkie rekordy z pliku CSV;

Wyznacz granice klas cenowych kwantylami z cen wszystkich rekordow;

Przypisz kazdy rekord do klasy **C1–C5**;

Oblicz srednia i odch. stand. kazdej cechy numerycznej;

foreach *rekord* **do**

if *dowolna cecha odbiega o* $> 3\sigma$ **then**

 Oznacz jako outlier;

end

end

Usun outliery ze zbioru;

Przetasuj zbior losowo (seed=42);

Podziel na zbior treningowy (80%) i testowy (20%);

Wybierz model (KNN / Naiwny Bayes / Drzewo decyzyjne);

Trenuj wybrany model na zbiorze treningowym;

foreach *przyklad ze zbioru testowego* **do**

 Przewidz klase cenowa;

 Porownaj z etykieta rzeczywista;

end

Wyswietl macierz pomylek i dokladnosc;

Algorithm 1: Glówny potok algorytmu programu

2.3 Baza danych

Dane wejściowe: plik `filtered_v3.csv` (ok. 15 MB, 200 000 rekordów). Zbiór został uzyskany poprzez przetasowanie i sztuczne zmniejszenie zbioru bazowego, pochodzącego z Kaggle, link: <https://www.kaggle.com/datasets/mrdaniilak/russia-real-estate-20182021> za pomocą skryptu z katalogu `datalab/`. Zawiera on około 5,5 miliona rekordów ogłoszeń sprzedaży mieszkań w Rosji w latach 2018 - 2021, redukowanych i tasowanych do 200 000 wpisów. Jako, że w plikach projektu nie załączono bazy, należy wygenerować ją samodzielnie z użyciem wspomnianego wyżej skryptu oraz pliku pobranego z linku.

2.4 Struktura rekordu

Kolumna	Typ	Opis
price	int	Cena mieszkania (RUB)
date	date	Data wystawienia ogłoszenia (YYYY-MM-DD)
time	time	Czas wystawienia ogłoszenia
geo_lat	float	Szerokość geograficzna
geo_lon	float	Długość geograficzna
region	int	ID regionu Rosji
building_type	int	Typ budynku (więcej informacji dostępnych na Kaggle)
level	int	Piętro mieszkania
levels	int	Liczba pięter w budynku
rooms	int	Liczba pokoi (-1 = kawalerka)
area	float	Powierzchnia całkowita (m ²)
kitchen_area	float	Powierzchnia kuchni (m ²)
object_type	int	1 = rynek wtórny, 11 = nowe budownictwo

2.5 Implementacja

Projekt podzielony jest na następujące pliki:

- `russian_flat.py` – klasy danych (`RussianFlatCsv`, `RussianFlatProps`, `RussianFlat`); wczytywanie i etykietowanie rekordów.
- `preprocessor.py` – usuwanie outlierów (reguła 3-sigma) i losowy podział na zbiory.
- `base_model.py` – abstrakcyjna klasa bazowa `BaseModel`.
- `knn.py` – klasyfikator KNN z hybrydową metryką i ważoną głosowaniem.
- `naive_bayes.py` – mieszany Naiwny Bayes (Gauss + Bernoulli + wielomianowy).
- `decision_tree.py` – drzewo decyzyjne CART z kryterium Gini.
- `data_analysis.py` – analiza statystyczna, macierz korelacji Pearsona, wykresy.
- `main.py` – główna petla menu, orkiestracja całego potoku.

Wybrane fragmenty kodu

Przykład: Wyznaczanie skal do normalizacji w KNN.

Listing 1: Obliczanie skal normalizacyjnych (`knn.py`)

```
1 def _compute_continuous_scales(self, training_data):
2     examples = [_ for _ in training_data]
3     columns = list(zip(*(
4         self._extract_features(flat.data)[0]
5         for flat in examples
6     )))
7     scales = []
8     for column in columns:
9         stdev = statistics.stdev(column) if len(column) > 1 else 0.0
10        if stdev <= 0.0:
11            stdev = 1.0
12        scales.append(stdev)
13    return tuple(scales)
```

Przykład: Trening Naiwnego Bayesa – wygładzanie wariancji dla toru gaussowskiego.

Listing 2: Wygładzanie wariancji (`naive_bayes.py`)

```
1 global_variances = []
2 global_columns = list(zip(*all_continuous_features))
3 for col in global_columns:
4     mean = sum(col) / len(col)
5     var = sum((x - mean)**2 for x in col) / (len(col) - 1)
6     global_variances.append(var)
7 epsilons = [max(1e-4 * var, 1e-9) for var in global_variances]
```


2.6 Eksperymenty

Poniżej przedstawiono wyniki przeprowadzonych eksperymentów, obejmujących trening przygotowanych klasyfikatorów oraz ich ocenę na zbiorze testowym.

KNN

Naiwny Bayes

Drzewo decyzyjne

Porównanie

2.7 Wnioski

Na podstawie wygenerowanych histogramów z nakładaną krzywą Gaussa stwierdzono:

- Cechy `area` i `kitchen_area` – po logarytmowaniu przybliżają rozkład normalny; traktowane gaussowsko.
- Cechy `geo_lat`, `geo_lon` – wyraźny rozkład wielomodalny (skupiska miast); traktowane jako kategorie przez spatial hashing.
- Cechy `publish_month` – rozkład cykliczny; traktowane jako kategorie.
- Cechy `level`, `levels`, `rooms` – wartości dyskretne; traktowane wielomianowo.

Na podstawie wyników eksperymentów stwierdzono:

- W praktyce, najlepszym z wytrenowanych klasyfikatorów okazał się Naiwny Bayes ze względu na swoją wysoką wydajność oraz skuteczność, a także brak problemów z treningiem, biorącym pod uwagę cały zbiór danych wejściowych.
- KNN ze względu na swoją bardzo niską wydajność, pomimo osiągnięcia najwyższej dokładności, w praktycznych zastosowaniach byłby bezużyteczny.

3 Pełen kod aplikacji

main.py

```
1 # Trzeba tu wrzucic wszystkie nasze pliki Pythona
2 print("Hello World!")
```

knn.py

```
1 print("Hello World!")
```
